

Lecture 2: Types + Expressions



INFO 1100: Introduction to Programming

Using Python

Using Python

Downloaded in **Lab 1!**

Using Python

interactive notebooks

Question 1

Much like I did in class, this reading argues that "learning to program means learning a new way to think." Give 2-3 examples from this chapter of ways in which you're being asked to think of familiar concepts in new ways.

[Write your answer here]

Question 2

Write 3 different expressions which result in the string `hello, hello, hello!`

```
1 # Your first expression
```

[1]

Python

Using Python

pure Python scripts

```
import os
import errno

bio_path = input("Author Last Name: ").lower()

if not os.path.exists(os.path.join("bios", bio_path + ".txt")):
    raise FileNotFoundError(errno.ENOENT, os.strerror(errno.ENOENT), bio_path)

with open(os.path.join("bios", bio_path + ".txt"), "r") as f:
    bio = f.read().strip()

print("")
```

Using Python

the “command line”

```
Last login: Tue Apr 21 10:31:09 on ttys002
[rebeccahicke@Rebeccas-MacBook-Pro book_covers % source ~/tldm/bin/activate
(tldm) rebeccahicke@Rebeccas-MacBook-Pro book_covers % python
Python 3.13.7 (main, Aug 14 2025, 11:12:11) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> print("hello world!")
hello world!
>>> █
```

Expressions

Expressions

an expression...

Expressions

an expression...

represents something

Expressions

an expression...

represents something

2.3

(3 * 7 + 2) * 0.1

“hello”

True

Expressions

an expression...

represents something

2.3

(3 * 7 + 2) * 0.1

“hello”

True

Python *evaluates* it

Expressions

an expression...

represents something

2.3

(3 * 7 + 2) * 0.1

“hello”

True

Python *evaluates* it

└ turns it into a **value**

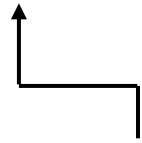
Types

Types

a set of values and operations on these values

Types

a set of values and operations on these values



describes what
kind of values
they are

Types

a set of values and operations on these values



e.g. +, -, /

Types

a set of values and operations on these values



depend on type

Types

a set of values and operations on these values

to determine type:

```
type (value)
```

Types

a set of values and operations on these values

to determine type:

```
type (2.0)
```

Types

a set of values and operations on these values

to determine type:

```
type(2.0)
```



```
<class 'float'>
```

Types

Types

`int`

Types

`int`

(integer)

Types

`int`

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

Types

`int`

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Types

`int`

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %
 ↑

multiplication

(integer)

same as the mathematical definition!

exponentiation

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

↑

division

(returns a float)

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

↑

floor division

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

↑

floor division

(returns an int)

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

↑

remainder

Types

int

(integer)

Values: ... - 3, -2, -1, 0, 1, 2, 3 ...

same as the mathematical definition!

Operations: +, -, *, **, /, //, %

↑

remainder

(modulo)

Types

float

Types

float

(floating point)

Types

float

(floating point)

Values: *approximations* of real numbers

(e.g. 2.8, 4.0, 6.6667)

Types

float

(floating point)

Values: *approximations* of real numbers

(e.g. 2.8, 4.0, 6.6667)

Python can't store most real numbers exactly!

Types

float

(floating point)

Values: *approximations* of real numbers

(e.g. 2.8, 4.0, 6.6667)

Operations: +, −, *, **, /, //, %

Types

float

(floating point)

Values: *approximations* of real numbers

(e.g. 2.8, 4.0, 6.6667)

Operations: +, −, *, **, /, //, %



same as for int

Types

`bool`

Types

bool

(boolean)

Types

bool

(boolean)

Values: **True** and **False**

Types

bool

(boolean)

Values: **True** and **False**

often come from comparing other values

$1 < 2$ $6 > 9$ $5 \leq 2$ $7 \geq 7$

$4 == 4$ $8 != 3$

Types

bool

(boolean)

Values: **True** and **False**

Operations: $<$, $>$, $\leq$, $\geq$, $==$, $!=$, not, and, or

Types

bool

(boolean)

Values: **True** and **False**

Operations: $<$, $>$, $\leq$, $\geq$, $==$, $!=$, not, and, or

Are not the same as the English words!

Types

str

Types

str

(string)

Types

str

(string)

Values: any sequence of characters

Types

str

(string)

Values: any sequence of characters

a sequence of characters in quotes

"Hello world!" and "Welcome to Info 1100!"

Types

str

(string)

Values: any sequence of characters

Operations: + (concatenation)

Types

str

(string)

Values: any sequence of characters

Operations: + (concatenation)

"Hello" + "world!" = "Helloworld!"

Types

str

(string)

Values: any sequence of characters

Operations: + (concatenation)

**notice that + means different things
for different types**

Converting between Types

Converting between Types

use type names as functions

Converting between Types

use type names as functions

`int(42.75)` → 42

Converting between Types

use type names as functions

`int(42.75)` → 42

always rounds down!

Converting between Types

use type names as functions

`int(42.75)` → 42

`float(66)` → 66.0

Converting between Types

use type names as functions

`int(42.75)` → 42

`float(66)` → 66.0

`str(127)` → "127"

Converting between Types

use type names as functions

`int(42.75)` → 42

`float(66)` → 66.0

`str(127)` → "127"

`int("3")` → 3

Converting between Types

"22" / 2

causes a **TypeError**!

Questions?

For tomorrow:

(06/24 @ 11:30am)

- Lab 2
- Readings
Think Python – Chapter 2
- GRQs